Sécurité logicielle (HAI821I)

Master Informatique
Département Informatique
Faculté des Sciences de Montpellier
Université de Montpellier

Examen du 21 mai 2025

Tous les documents de cours sont autorisés. L'examen dure 2h. Le barème est donné à titre indicatif. Le sujet comporte 2 pages et il y a 3 exercices.

Exercice 1 (6 pts)

On souhaite rajouter les expressions de post/pré-incrémentation/décrémentation à notre ensemble d'expressions avec la syntaxe suivante :

$$e \quad ::= \quad \ldots \mid x{+}{+} \mid {+}{+}x \mid x{-}{-} \mid {-}{-}x$$

Intuitivement, l'évaluation de $x{+}{+}$ dans un environnement E rend la valeur de x dans E, puis incrémente la valeur de x dans E. En revanche, l'évaluation de ${+}{+}x$ incrémente la valeur de x dans E, puis rend la (nouvelle) valeur de x dans E. Le fonctionnement est similaire pour $x{-}{-}$ et ${-}{-}x$ mais en post-décrémentant ou en pré-décrémentant. Ces nouvelles expressions font donc, dans tous les cas, un effet de bord (sur E).

1. Donner les règles d'évaluation des expressions de post/pré-incrémentation/décrémentation dans la sémantique à grands pas. Vous ferez attention à bien définir, au préalable, la relation d'évaluation, qui ne retourne plus seulement des valeurs mais aussi des contextes d'évaluation (pour tenir compte des effets de bord éventuels).
2. Évaluer les expressions suivantes dans la sémantique à grands pas :
   (a) $(x{+}{+}) + ({-}{-}y)$ dans l'environnement $E = (x, 1), (y, 2)$.
   (b) $(x{+}{+}) + x$ dans l'environnement $E = (x, 1)$.
3. Démontrer que les instructions "$y := x{+}{+}$" et "$y := x; x := x + 1$" sont équivalentes dans la sémantiques à grands pas.

Exercice 2 (7 pts)

On considère le langage des expressions arithmétiques et booléennes (sans variable et sans opération de comparaison) vu en cours :

$$e \quad ::= \quad n \mid e_1 + e_2 \mid e_1 - e_2 \mid e_1 \times e_2 \mid e_1/e_2 \mid \text{true} \mid \text{false} \mid \text{not}(e) \mid e \text{ and } e \mid e \text{ or } e$$

Nous munissons ce langage d'un système de types comme vu en cours également :

$$\tau \quad ::= \quad \text{int} \mid \text{bool}$$

Les règles de typage de ce langage sont les suivantes :

$$\frac{}{\Gamma \vdash n : \text{int}}\ \tau_{\mathbb{Z}} \qquad \frac{(x,\tau) \in \Gamma}{\Gamma \vdash x : \tau}\ \tau_{\mathbb{V}} \qquad \frac{\Gamma \vdash e_1 : \text{int} \qquad \Gamma \vdash e_2 : \text{int}}{\Gamma \vdash e_1 \text{ op } e_2 : \text{int}}\ \tau_{\text{op}},\ \text{avec op} \in \{+,-,\times,/\}$$

$$\frac{}{\Gamma \vdash \text{true} : \text{bool}}\ \tau_{\text{true}} \qquad \frac{}{\Gamma \vdash \text{false} : \text{bool}}\ \tau_{\text{false}}$$

$$\frac{\Gamma \vdash e : \text{bool}}{\Gamma \vdash \text{not}(e) : \text{bool}}\ \tau_{\text{not}} \qquad \frac{\Gamma \vdash e_1 : \text{bool} \qquad \Gamma \vdash e_2 : \text{bool}}{\Gamma \vdash e_1 \text{ op } e_2 : \text{bool}}\ \tau_{\text{op}},\ \text{avec op} \in \{\text{and}, \text{or}\}$$

1. Donner les règles d'évaluation de ce langage dans la sémantique à petits pas. Étant donné qu'il n'y a pas de variable, il n'y aura pas d'état E dans les règles de cette sémantique.
2. Démontrer le théorème suivant : pour toutes expressions e_1 et e_2, et tout type τ, si $e_1 : \tau$ et $e_1 \to^* e_2$ alors $e_2 : \tau$.
   Ce théorème, généralement appelé « subject reduction », exprime que la relation sémantique préserve les types. La preuve se fera par induction sur une des deux relations en hypothèse, à savoir $e_1 : \tau$ ou $e_1 \to^* e_2$ (à vous de deviner laquelle).

Exercice 3 (7 pts)

En utilisant la sémantique axiomatique (logique de Hoare), démontrer la validité du triplet suivant (ce qui revient à démontrer que le programme ci-dessous implante la fonction qui calcule la somme des n premiers entiers) :

$\{\}$
$i := 0;$
$r := 0;$
while i != n do
  $i := i + 1;$
  $r := r + i;$
$\{r = \frac{n(n+1)}{2}\}$